

Overspeed Vehicle Detection System

-Jassmitha Jammu

1. Project Overview

This project implements an **Overspeed Vehicle Detection System** that processes a traffic video, detects vehicles, estimates their speed, and flags overspeeding vehicles. The system provides a simple web interface where users can upload a video and view the processed output.

The system integrates:

- Computer Vision
- Deep Learning (Object Detection)
- Basic Tracking Logic
- Web Interface for user interaction

2. Problem Statement

To design and implement a system that:

- Detects vehicles from a video stream
- Estimates their speed using frame-based motion
- Identifies vehicles exceeding a predefined speed limit
- Displays results visually in a processed video output

3. System Architecture

Workflow

1. User uploads a video via web interface

2. Backend receives the file
3. Video is processed frame-by-frame
4. Vehicles are detected using a deep learning model
5. Speed is estimated using frame differences
6. Overspeeding vehicles are flagged
7. Processed video is returned and displayed

4. Technologies Used

Programming Language

- Python 3.10 (recommended for compatibility)

Libraries & Frameworks

- OpenCV

Used for:

- Video capture and frame processing
- Drawing bounding boxes and annotations
- Ultralytics YOLO

Used for:

- Real-time object detection
- PyTorch

Used as:

- Backend deep learning framework
- Flask

Used for:

- Web interface and routing
- NumPy

Used for:

- Numerical operations

5. Model Details

Object Detection Model

- Model: YOLOv8 (Nano variant)
- File: yolov8n.pt

Why YOLO?

- Real-time detection capability
- High efficiency with low computational cost
- Suitable for video-based applications

6. Model Architecture (Conceptual)

YOLO (You Only Look Once) is a **single-stage object detector**.

Key Components:

- Convolutional layers (feature extraction)
- Detection head (bounding box prediction)
- Anchor-free prediction (in YOLOv8)

Activation Functions Used:

- **SiLU (Sigmoid Linear Unit)**

Formula:

$$f(x) = x * \text{sigmoid}(x)$$

Why SiLU:

- Smooth non-linearity
- Better gradient flow than ReLU

7. Vehicle Detection Logic

From COCO dataset classes:

Class ID	Vehicle Type
2	Car
3	Motorcycle
5	Bus
7	Truck

Only these classes are considered.

8. Speed Estimation Logic

Speed is estimated using **frame-based displacement**.

Steps:

1. Two horizontal lines are defined:
 - a. Entry line (`line_y1`)
 - b. Exit line (`line_y2`)
2. When a vehicle crosses:
 - a. First line → entry frame recorded
 - b. Second line → exit frame recorded
3. Speed calculation:

Speed = Distance/Time

Where:

- Distance = pixel difference between lines
- Time = (frame difference / FPS)

Scaled formula used:

$\text{speed} = (\text{distance_pixels} / \text{time_taken}) * 0.1$

9. Tracking Method

A simple tracking mechanism is used:

`obj_id = int(cx / 100)`

Where:

- cx = center x-coordinate of bounding box

Limitations:

- Not robust for overlapping vehicles
- Not suitable for real-world deployment

10. Frontend Design

Features:

- Black background UI
- Centered upload interface
- Blue-themed styling
- Video output displayed on same page

Technologies:

- HTML

- CSS (inline styling)

11. Backend (Flask)

Responsibilities:

- Accept file upload
- Save input video
- Call processing function
- Return processed video

12. Project Structure

```
vehicle_violation_project/  
|  
├─ app.py  
├─ vehicle.py  
├─ templates/  
|   └─ index.html  
├─ uploads/  
├─ outputs/  
├─ requirements.txt  
└─ README.md
```

13. Dependencies

requirements.txt

```
opencv-python==4.9.0.80  
ultralytics==8.2.0  
torch==2.2.2  
torchvision==0.17.2
```

```
numpy<2  
flask==3.0.0
```

14. How to Run the Project

Step 1: Clone Repository

```
git clone <repo-url>  
cd vehicle_violation_project
```

Step 2: Create Virtual Environment

```
python -m venv venv  
venv\Scripts\activate
```

Step 3: Install Dependencies

```
pip install -r requirements.txt
```

Step 4: Run Application

```
python app.py
```

Step 5: Open in Browser

<http://127.0.0.1:5000>

15. How to Use

1. Upload a traffic video
2. Click “Upload & Detect”
3. Wait for processing
4. View output video with:
 - a. Bounding boxes
 - b. Overspeed alerts

16. Output

The system generates:

- Processed video (output .mp4)
- Visual annotations:
 - Vehicle boxes
 - Overspeed warnings

17. Limitations

- No real-world speed calibration
- Basic tracking (not robust)
- No number plate recognition
- Processing is not real-time in UI

18. Conclusion

This project demonstrates an end-to-end pipeline combining:

- Deep learning for detection
- Computer vision for tracking
- Web interface for usability

It successfully detects and flags overspeeding vehicles from video input, providing a functional prototype for intelligent traffic monitoring systems.